Taller Ingeniería de Software II

Mariana Duque Giraldo - mduquegi@unal.edu.co

Docente:

Jose Albeiro Montes Gil - joamontesgi@unal.edu.co



Universidad Nacional de Colombia
Facultad de Administración
Administración de Sistemas informáticos
2026

Escenarios

Escenario de Carga:

Está definido en LoadUser:

```
class LoadUser(ApiUser):
    """Carga nominal: usuarios tranquilos y pausas cortas entre solicitudes."""

    weight = 3
    wait_time = between(1, 3)
```

Este escenario:

- Representa la carga normal esperada
- Tiene pausas de 1 a 3 segundos entre solicitudes
- Tiene mayor peso dentro de la ejecución

Se puede ejecutar con:

```
locust -f locustfile.py --user-classes LoadUser --headless -u 30 -r 3 -t 10m
```

Escenario de estrés:

Está definido en `StressUser`:

```
class StressUser(ApiUser):
    """Estrés: genera más solicitudes por usuario para encontrar el quiebre."""

    weight = 1
    wait_time = between(0.2, 0.8)
```

Este escenario:

- Hace solicitudes con pausas muy cortas
- Busca encontrar el punto donde la API comienza a degradarse
- Puede producir mas solicitudes por segundo

Se puede ejecutar con:

```
locust -f locustfile.py --user-classes StressUser --headless -u 200 -r 20 -t 10m
```

Escenario de Capacidad

Está definido en `CapacityUser`:

```
class CapacityUser(ApiUser):
    """Capacidad: mantiene una carga constante durante una ejecución larga."""

    weight = 1
    wait_time = between(1, 2)
```

Este escenario:

- Mantiene una carga estable
- Está pensado para ejecutarse durante bastante tiempo
- Permite observar degradación gradual, problemas de memoria o conexiones

Se puede ejecutar con:

```
locust -f locustfile.py --user-classes CapacityUser --headless -u 50 -r 5 -t 60m
```

Las tareas que usan todos los escenarios

Los tres escenarios heredan de `ApiUser`, donde están definidas las solicitudes HTTP:

- `list_users` — `GET /api/users`
- `list_emails` — `GET /api/users/emails`
- `list_adults` — `GET /api/users/over-twenty`
- `create_users` — `POST /api/users/bulk`

La mezcla de tareas esta definida con los pesos:

```
@task(5)  # usuarios
@task(3)  # emails
@task(2)  # mayores de 20
@task(1)  # creación masiva
```

Parámetros

Configuración General

Parámetro	Valor
Herramienta	Locust 2.46.7
Host	`http://127.0.0.1:8000`
Archivo	`locustfile.py`
API	Laravel 9
Base de datos	laravel MySQL
Registros	1.500.000 usuarios
Tamaño de página	50 registros
Paginación máxima permitida	200 registros
Dataset	API con paginación habilitada

Escenario de carga

Parametro	Valor
Clase de usuario	`LoadUser`
Usuarios	30
Spawn rate	3 usuarios/segundo
Duración	10 minutos
Tipo de prueba	Carga nominal
Tiempo entre solicitudes	Entre 1 y 3 segundos

Escenario de estrés

Parámetro	Valor
Clase de usuario	`StressUser`
Usuarios	200
Spawn rate	20 usuarios/segundo
Duración	10 minutos
Tipo de prueba	Estrés
Tiempo entre solicitudes	Entre 0,2 y 0,8 segundos

Escenario de capacidad

Parametro	Valor
Clase de usuario	`CapacityUser`
Usuarios	50
Spawn rate	5 usuarios/segundo
Duracion	60 minutos
Tipo de prueba	Capacidad o Soak test
Tiempo entre solicitudes	Entre 1 y 2 segundos

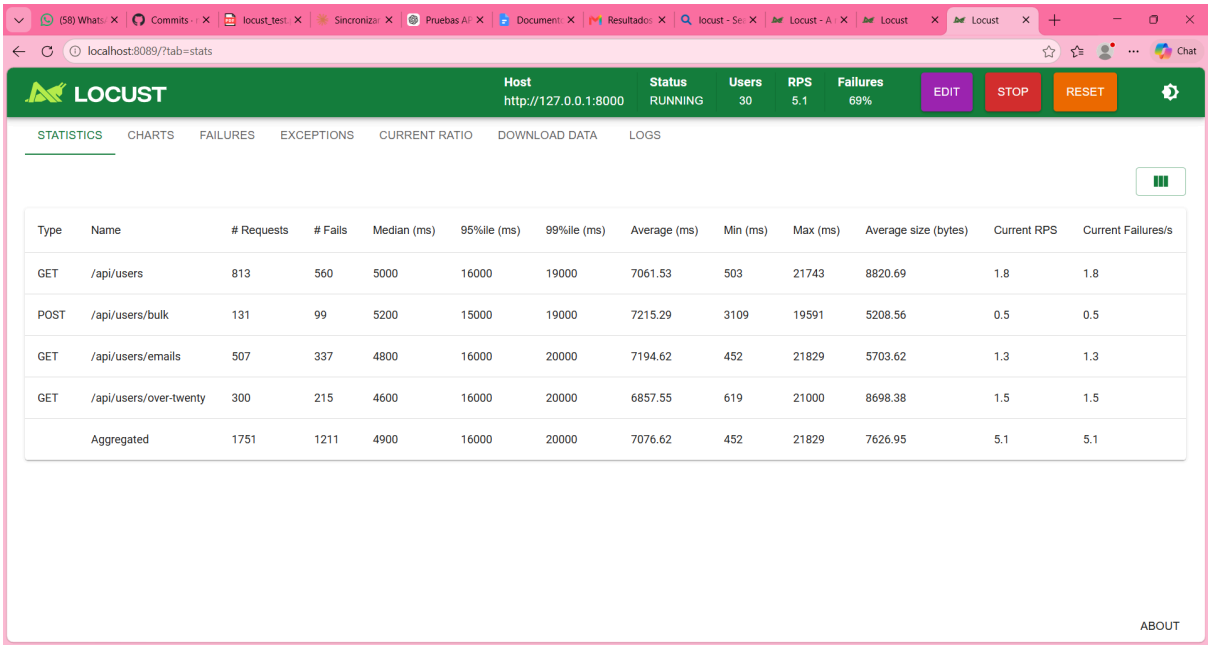
Operaciones evaluadas

La mezcla de solicitudes definidas en `locustfile.py`

Endpoint	Peso	Propósito
`GET /api/users`	5	Listar usuarios paginados
`GET /api/users/emails`	3	Consultar identificadores y correos
`GET /api/users/over-twenty`	2	Consultar usuarios mayores de 20 años
`POST /api/users/bulk`	1	Crear 3 usuarios simultáneamente

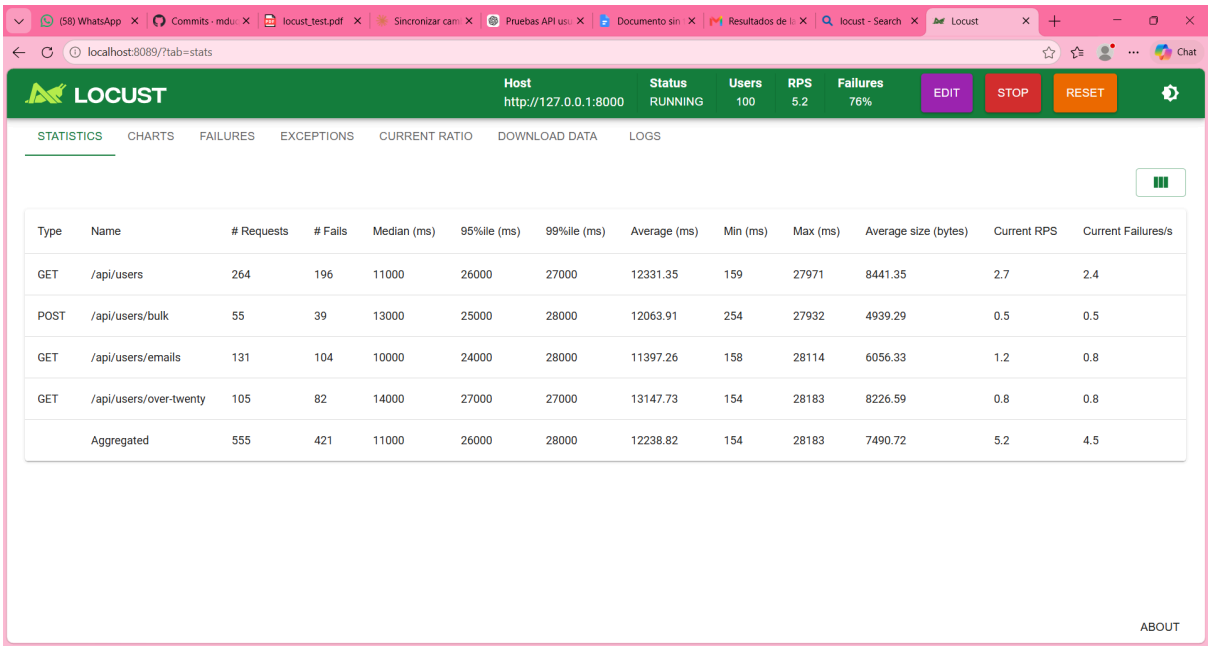
Pruebas

Carga - LoadUser



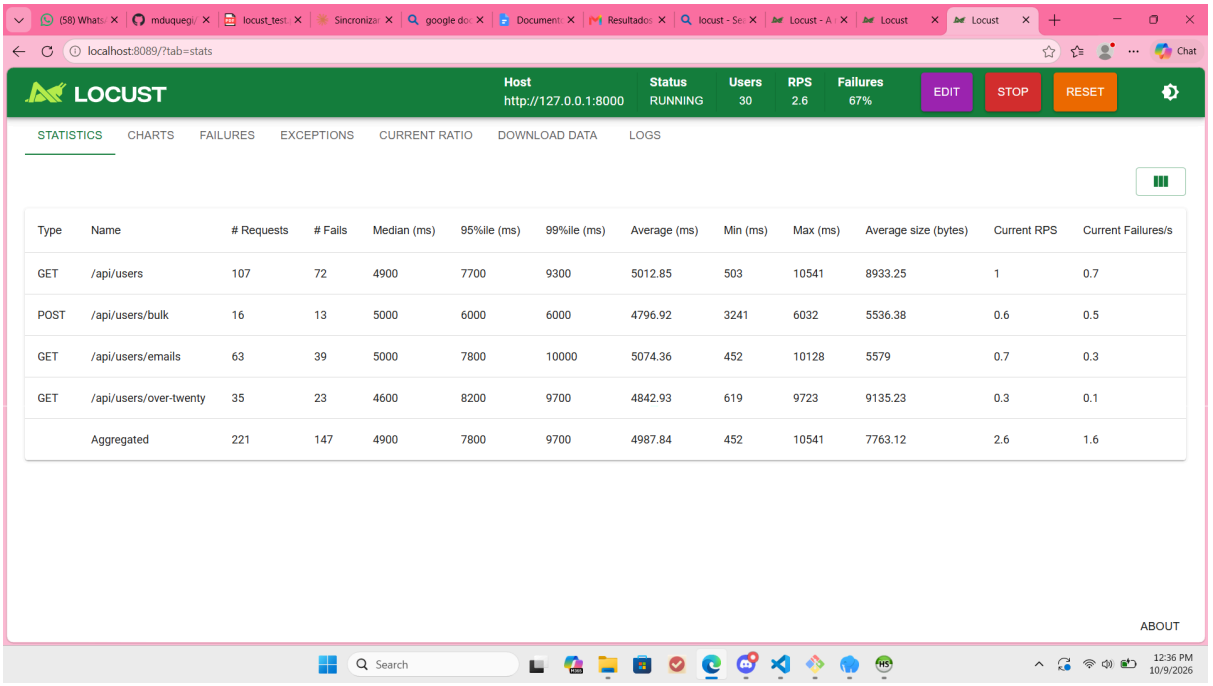
Type	Name	# Requests	# Fails	Median (ms)	95%ile (ms)	99%ile (ms)	Average (ms)	Min (ms)	Max (ms)	Average size (bytes)	Current RPS	Current Failures/s
GET	/api/users	813	560	5000	16000	19000	7061.53	503	21743	8820.69	1.8	1.8
POST	/api/users/bulk	131	99	5200	15000	19000	7215.29	3109	19591	5208.56	0.5	0.5
GET	/api/users/emails	507	337	4800	16000	20000	7194.62	452	21829	5703.62	1.3	1.3
GET	/api/users/over-twenty	300	215	4600	16000	20000	6857.55	619	21000	8698.38	1.5	1.5
	Aggregated	1751	1211	4900	16000	20000	7076.62	452	21829	7626.95	5.1	5.1

Estrés - StressUser



Type	Name	# Requests	# Fails	Median (ms)	95%ile (ms)	99%ile (ms)	Average (ms)	Min (ms)	Max (ms)	Average size (bytes)	Current RPS	Current Failures/s
GET	/api/users	264	196	11000	26000	27000	12331.35	159	27971	8441.35	2.7	2.4
POST	/api/users/bulk	55	39	13000	25000	28000	12063.91	254	27932	4939.29	0.5	0.5
GET	/api/users/emails	131	104	10000	24000	28000	11397.26	158	28114	6056.33	1.2	0.8
GET	/api/users/over-twenty	105	82	14000	27000	27000	13147.73	154	28183	8226.59	0.8	0.8
	Aggregated	555	421	11000	26000	28000	12238.82	154	28183	7490.72	5.2	4.5

Capacidad - CapacityUser



Type	Name	# Requests	# Fails	Median (ms)	95%ile (ms)	99%ile (ms)	Average (ms)	Min (ms)	Max (ms)	Average size (bytes)	Current RPS	Current Failures/s
GET	/api/users	107	72	4900	7700	9300	5012.85	503	10541	8933.25	1	0.7
POST	/api/users/bulk	16	13	5000	6000	6000	4796.92	3241	6032	5536.38	0.6	0.5
GET	/api/users/emails	63	39	5000	7800	10000	5074.36	452	10128	5579	0.7	0.3
GET	/api/users/over-twenty	35	23	4600	8200	9700	4842.93	619	9723	9135.23	0.3	0.1
	Aggregated	221	147	4900	7800	9700	4987.84	452	10541	7763.12	2.6	1.6